Forkable sandboxes let computers explore executable alternatives.

Self-organization → self-driving cars → self-driving compute.

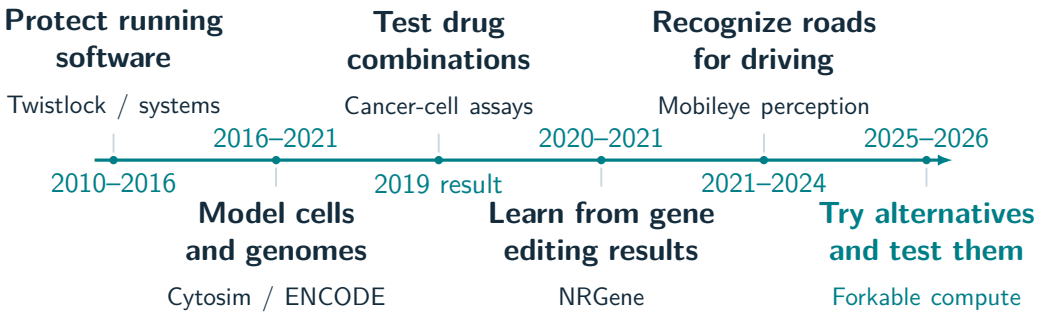


Reuse reached state. Run alternatives. Verify the result.

Yossi Eliaz
Principal Engineer, Incredibuild
Senior Lecturer, HIT

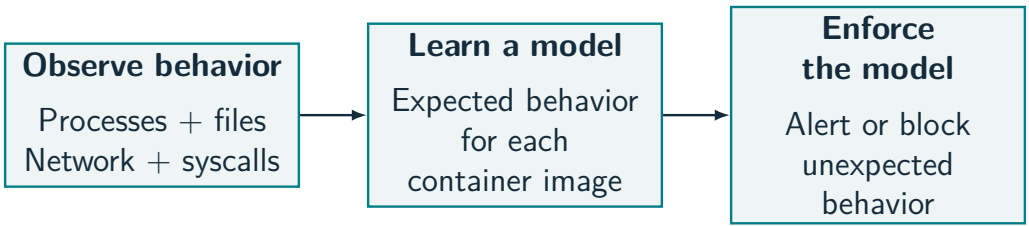
Cambridge Systems Research Group, 15 October 2026. Research and engineering perspectives; employer affiliation disclosed.

My work led from securing software and modeling cells to self-driving compute.



Career periods; cancer-drug result (2019). Work periods overlap.

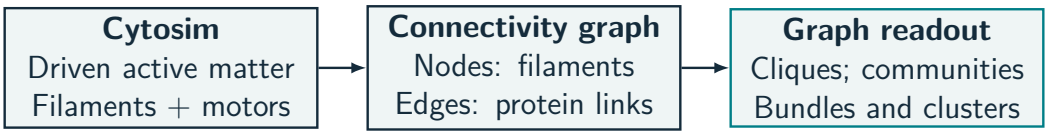
Twistlock learned container behavior and enforced the resulting model.



Vendor-reported learning plateau: usually about **1 hour** of cumulative runtime

Twistlock 1.7 Tech News, January 2017. Product/team mechanism.

Graph measures link protein binding to actomyosin self-organization.



2–7 binding sites per linker **600 s** simulated time **30** random starts

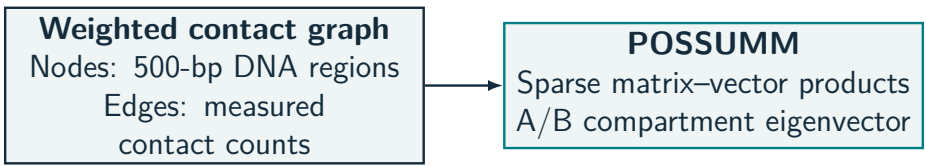
More binding sites promoted bundles and larger clusters.

Biological motivation: contraction and dendritic-spine plasticity involved in memory.

Eliaz, Nedelec, Morrison, Levine & Cheung, Phys. Rev. E 102:062420 (2020).

POSSUMM computed 500-bp chromosome compartments with 23 GB of RAM.

Hi-C is a weighted contact graph between DNA regions.



23 GB RAM
Measured

2.5 min
Measured

> 4.6 **TB RAM**
Dense projection

Chromosome 1, 500 bp. A/B identifies active and inactive chromatin.

Harris et al., including Eliaz, Nature Communications 14:3303 (2023), Box 1 and Supplementary Fig. 2.

ENCODE recorded how each result came from its input data.

14,000+ datasets

DNA read files	→	Repeatable analysis	→	Traceable results
Input data for each experiment		Workflow + software environment		Which data? Which software? Quality checks

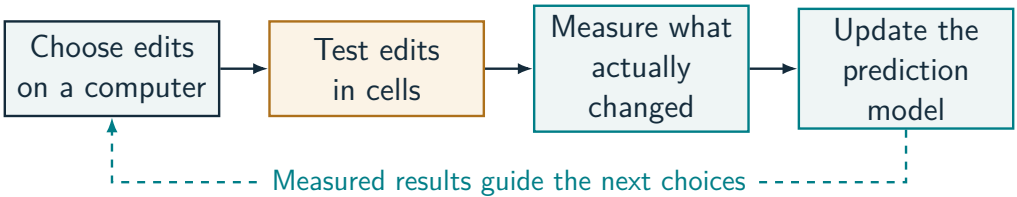
Each result records its inputs, software and checks.

Hitz et al., including Eliaz, The ENCODE uniform analysis pipelines (2023). Coauthored infrastructure.

PUBLISHED

Computer predictions guide gene-editing experiments, then learn from them.

CRISPR: a tool for editing DNA. Models help choose edits to test.



Pre-lab check (OffRisk): **118 predicted DNA matches**, including **4 unintended sites** flagged for higher priority.

CRISPR-IL / GoGenome: Noam Barkai & Yossi Eliaz, AWS (2022). Separate OffRisk example: Barkai, Malul, Eliaz et al. (2023).

PUBLISHED

Researchers chose a drug pair to test.

Lab tests in mouse breast-cancer cells.



The pair kept cell killing and reduced the PD-L1 rise.

PD-L1 can help cancer cells avoid immune attack.

10 drugs $\longrightarrow$ 45 possible pairs

At one dose per drug. Researchers did not test all 45 pairs.

Abemaciclib slows cell division (CDK4/6). SI-2 blocks a growth regulator (SRC-3).

Gilad, Eliaz et al., Scientific Reports 9:15099 (2019), Figs. 1, 3 and S2. E0771 cell model.

300 million cars build Mobileye's Roadbook.



Mobileye autonomous test vehicle (AV).
Manhattan, June 2021.
My work: junction perception, 2021–2024.

Cars worldwide
Send anonymized road data

↓

Match the same road
Combine repeated observations

↓

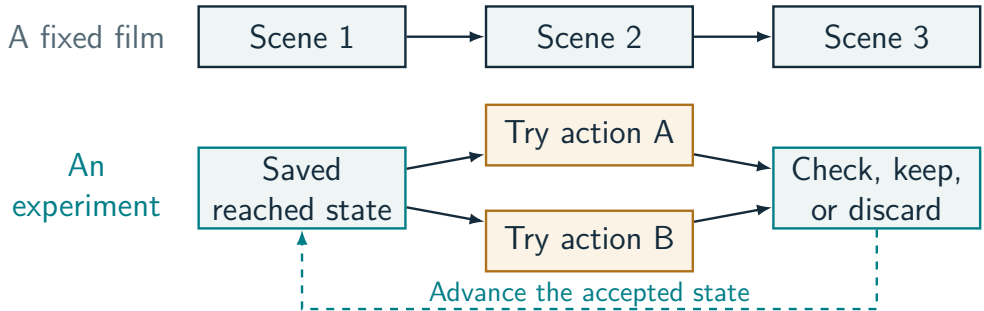
HD Roadbook
Send the updated map back to cars

Roadbook combines lanes, signs and driving paths. HD means high definition.

2025 mapping: **34 billion road-miles** ≈ **1,080 road-miles/second** (annual average).

300M fleet: speaker-reported. 2025 mapping data. Roadbook workflow. Photo: Mobileye press kit.

Forkable computation lets us try different futures from the same state.



Organizing analogy and proposed execution model. Stored-program hardware remains the substrate.

The forkable API prepares once and runs alternatives in private children.

Try the same three edits, run the same checks, select a result.

Python loop

```
out = []
for edit in edits:
    env = prepare()
    env.apply(edit)
    out.append(test(env))
best = select(out)
```

3 prepared environments.

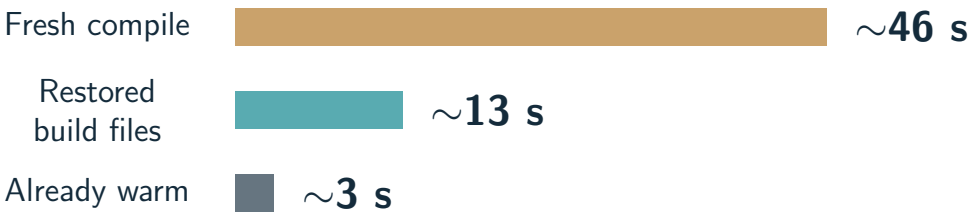
Forkable API

```
parent = checkpoint(prepare())
children = fork(parent, n=3)
execute(children, edits)
out = evaluate(children, test)
best = select(out)
```

1 preparation; 3 private children.

Proposed API sketch. Both columns are Python; each check returns its artifact and evidence.

Caching lets repeated experiments reuse their expensive build work.

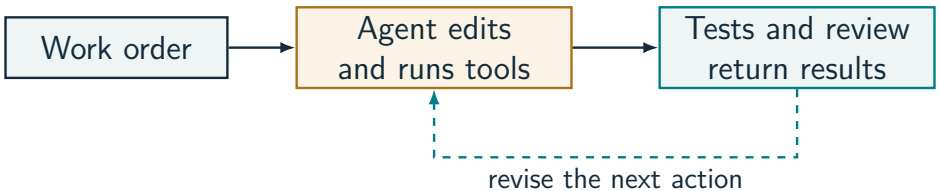


Restored cache: copy saved build files into a fresh runner.
Hot cache: reuse build files already available from a previous build.

Tokio example: restoring the cache saved about 33 seconds per compile.

Eliaz, Tokio issue #8200, June 2026: external Incredibuild demo. Artifact caching.

An agent changes the environment that executes its next experiment.



Airflow Factory: sandboxes through Docker Sandboxes (`sbx`); hosted models through Databricks.

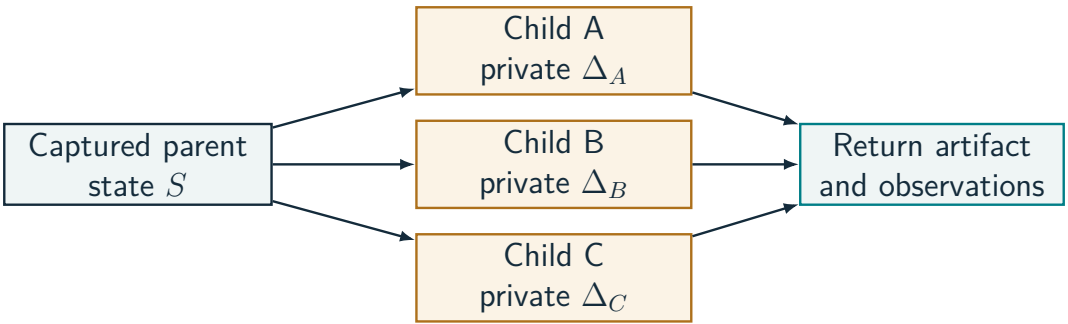
A sandbox is a policy boundary; VMs and containers are implementation choices.

Term	What it provides	Example
VM	Separate guest kernel	KVM + QEMU
microVM	Guest kernel; fewer devices	Firecracker
Container	Processes share a Linux kernel	Namespaces; cgroups
Sandbox	Policy over access and actions	Container or VM
cgroups	Resource usage and limits	CPU, memory, I/O

Docker Sandboxes (`sbx`): microVM + private Docker daemon.

Docker Sandboxes architecture; Firecracker; Linux cgroup v2.

A fork creates private continuations from one captured execution state.



Execution abstraction; backend-specific state and external-effect semantics.

Different fields use the same pattern: explore alternatives, then check the results.

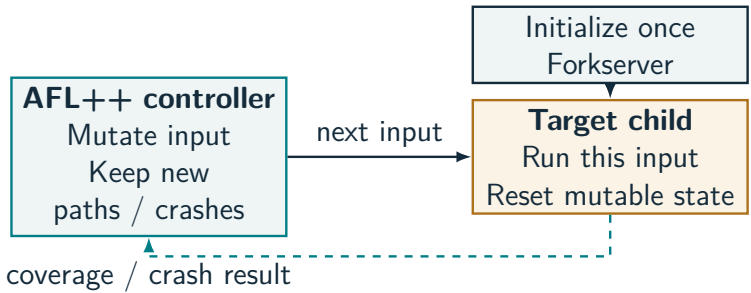
The common pattern: shared setup → different trials → gather results.

Security and AI	Changed inputs, model variants and candidate programs
Genomics and physics	Gene perturbations, drug partners and orbit connections
Our software factory	Repairs, tests and delivery of the resulting patch

The examples show exploration and gathering; the runtime chapter turns the pattern into an execution contract.

AFL++ finds bugs by changing inputs and reusing the initialized program.

AFL++ is a fuzzer: it tests changed inputs and follows new code paths.



Persistent mode: about 1,000 inputs per child, then start a fresh child.
AFL++; persistent-mode documentation.

AFL++ reports 10–20 times faster execution in persistent mode.

10–20×

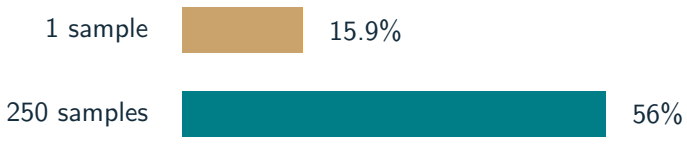
Typical speedup reported for persistent mode

One child amortizes process creation across many inputs.

Correct per-input reset keeps the reused state useful.

AFL++ documentation; published typical gain, not our benchmark.

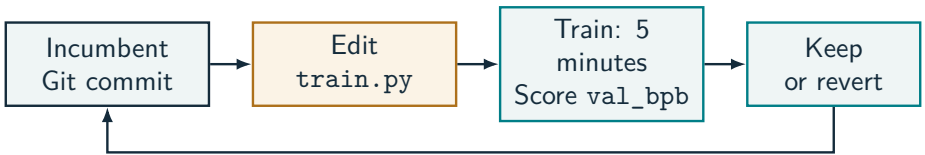
250 samples raised SWE-bench Lite coverage from 15.9% to 56%.



Coverage means a correct patch exists; a selector must recognize it.

Brown et al., Large Language Monkeys (2024); SWE-bench Lite.

Autoresearch kept 23 experiments and lowered validation bits per byte.

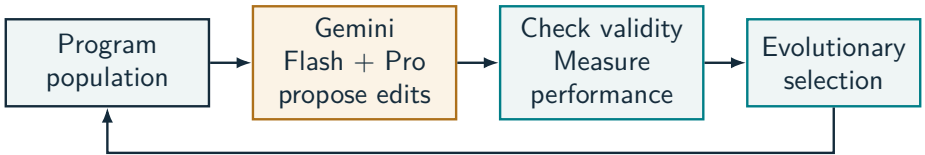


126 attempts: 23 kept, 102 discarded, 1 crash.

$0.997900 \rightarrow 0.969686$: a 2.8% reduction; lower is better.

Karpathy, autoresearch protocol; reported session.

AlphaEvolve recovered 0.7% of Google's worldwide compute.

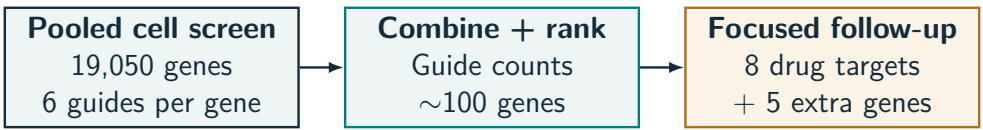


A deployed Borg scheduling heuristic recovered 0.7% on average.

Google DeepMind, AlphaEvolve (2025); production application.

Over 120,000 CRISPR guides helped rank about 100 candidate genes.

3 conditions $\times$ 3 repeat cultures $=$ **9 pooled arms**.



Each guide targets a DNA site. Drug-treated and control cultures reveal changes in survival.

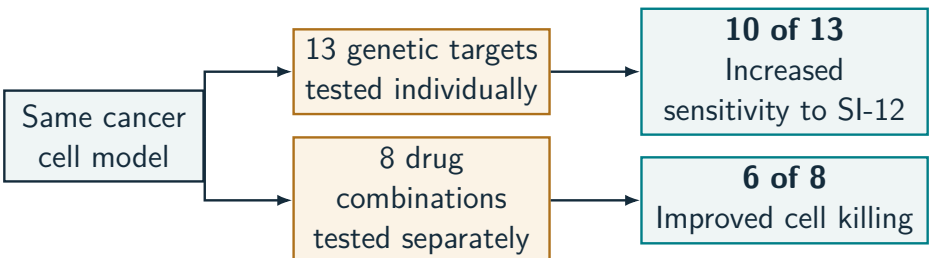
One ranking rule, DRACO, requires 4 of 6 guides to agree in direction.

At the gene level: $19{,}050/{\sim}100 \approx 190$, roughly **190-fold narrowing**.
SI-12 blocks SRC-3, a protein that supports cancer-cell growth.

Gilad, Eliaz et al., Communications Biology 4:399 (2021). Ranking nominates candidates for validation.

Focused tests found promising gene targets and drug combinations.

MCF-7: human breast cancer cells grown in the laboratory.

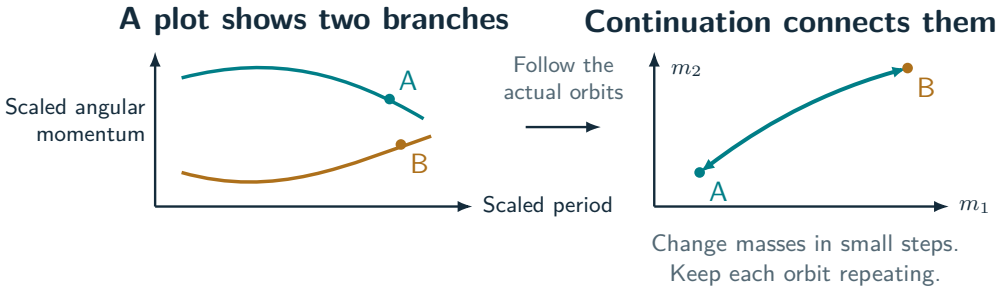


Compare separate trials with the same readout.

Gilad, Eliaz et al. (2021), Fig. 3. ≥ 4 technical repeats per point; ≥ 2 independent experiments per plot. Cell assays.

With Chamo, we connected two apparent branches of one orbit family.

Li, Li & Liao (2021): **one family**. Stančević et al. (2023): **two plotted branches**.



Two branches in a projection can belong to one connected family of solutions.

With **Ori Chamo**. Schematic of Fig. 1, Chamo–Eliaz preprint (2026). The 135,445-orbit catalog is the sampled scope.

MEASURED

All 26 selected connections survived checks in both directions.

Broad gaps	●●●●●	5 / 5
Chart jumps	●●●●●●●●●●●●●●●●●●●●	20 / 20
Distant pair	●	1 / 1

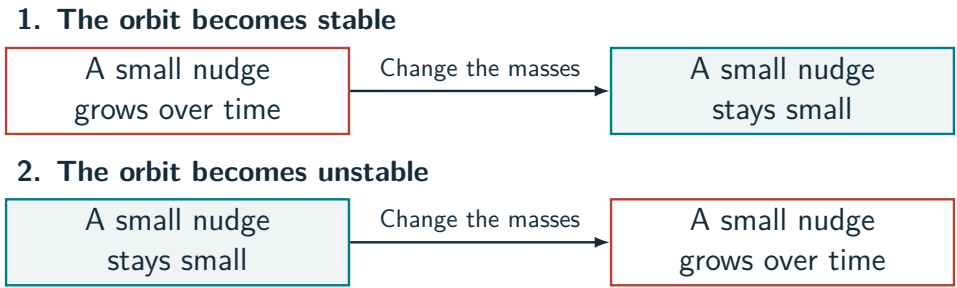
The selected links connect within **one sampled solution component**.

Representative stability changes were independently checked at **60-digit precision**.

Chamo–Eliaz preprint; Atlas numerical records.

An orbit can gain or lose stability.

Stable: a small nudge stays small. Unstable: it grows.



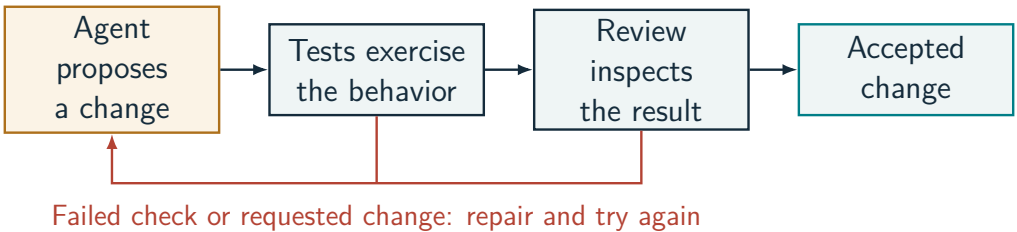
Two examples in the same orbit family.

Independent calculations using 60-digit arithmetic found the same changes.

Simplified Fig. 2, Chamo–Eliaz preprint (2026). Small disturbances within the orbital plane (**linear, planar** stability).

Software agents improve code through a feedback loop.

Example goal: let a sandbox install Python tools.

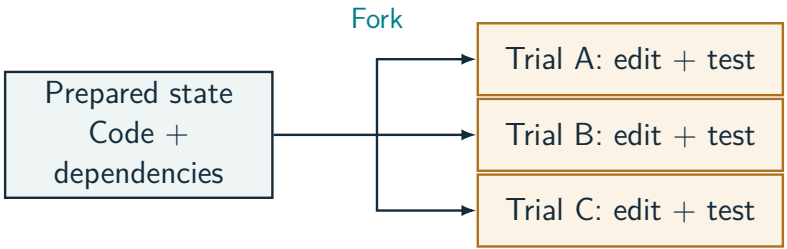


Checks determine which change the next round keeps.

Observed software-factory workflow. The recorded repair ran serially.

Forking tests different ideas from the same prepared state.

Fan-out: create private trials from one shared start.

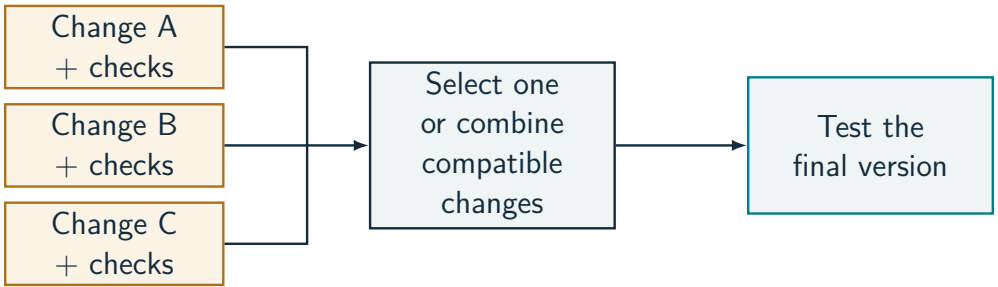


Each trial explores a different next step in its own copy.

Proposed parallel design. The recorded factory repair used a serial loop.

Fan-in checks the version we choose or combine.

Fan-in: collect changes together with their test results.

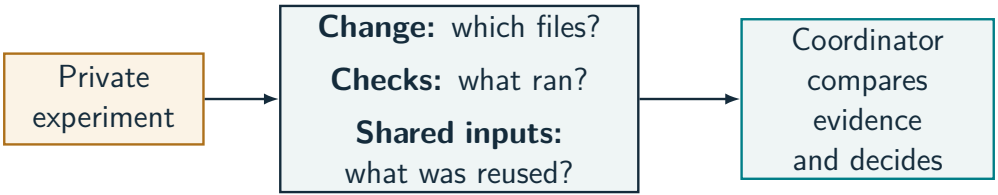


A merged change can create interactions that the separate trials never tested.

Proposed code-result flow. Selection and composition require checks of the exact retained version.

Our proposal connects state reuse with evidence about returned results.

Each trial returns enough information to judge what it produced.

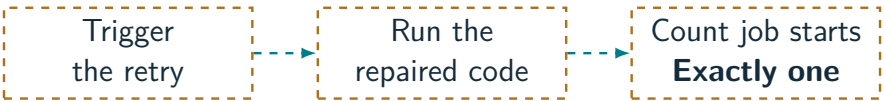


The proposed interface carries the result and its evidence into the acceptance decision.

Forking, testing and review are established tools. The contribution is their connected execution contract.

A useful test checks the behavior that the change promises.

Example: retrying a job should start it **exactly once**.



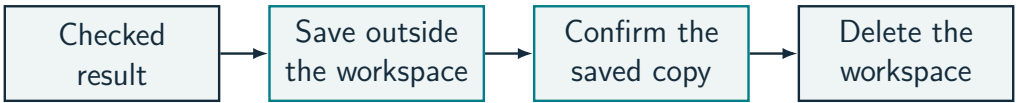
Our review approved a repair while this targeted check was still missing.

Many passing tests can still miss the behavior that changed.

Observed coverage gap. The diagram shows the required test.

Accepted work must survive the experiment.

Save the result and its test records beyond the temporary workspace.



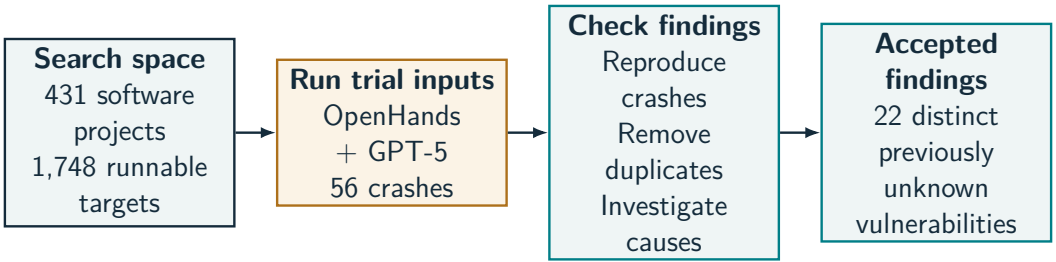
A separate run lost an approved patch during cleanup.

Saving preserves the work. Publishing requires its own approval.

Observed loss. This ordering is a proposed recovery rule.

AI testing produced 56 crashes and 22 confirmed new vulnerabilities.

An AI agent wrote and ran test inputs against current code.



A crash is a lead. Reproduce it and remove duplicates before counting a new vulnerability.

Berkeley CyberGym report, latest-code GPT-5 campaign.

A forkable runtime must preserve state, gather evidence and control acceptance.

The examples are complete. Now we turn their pattern into a system.

Run	What state is reused, and when is reuse cheaper?
Gather evidence	What ran, and how much information do its results add?
Accept	Which checked result may continue or be published?

The following chapters cover runtime costs and boundaries, information aggregation, then acceptance.

The next action determines which state a continuation needs.

Next action	Mechanism	State received
Edit source or build	Worktrees and build cache	Files and cached inputs
Continue live execution	Process or VM checkpoint	Supported memory and runtime state
Reuse an observation	Replay	Recorded execution or response

CRIU, gVisor, Firecracker and rr documentation; SnowFlock, EuroSys 2009; DeepSeek-V4 §5.2.5.

MEASURED

Our paper recorded 256 sandbox creates with a 3.44-second median.

Operation	Recorded result
Default sandbox environment	**256 / 256 creates completed**
Concurrency requested	**8 calls at a time**
Creation API latency	**Median 3.44 s; p95 9.00 s**

p95 means **95% of recorded calls completed within 9.00 seconds**.

Eliaz, arXiv:2607.09689v4, Table 2. Creation API round trip; teardown excluded.

PUBLISHED

Isorun led the October 2 sandbox startup benchmark at a 73 ms median.

Time to interactive: create a sandbox and run its first command.

Provider	Median	p95	Success
Isorun	**73 ms**	**78 ms**	**100 / 100**
Miosa	163 ms	188 ms	100 / 100
Archil	238 ms	250 ms	100 / 100

ComputeSDK: 100 concurrent requests per provider, 2 October 2026.

Cheap starts raise the bar for state reuse.

ComputeSDK Burst TTI and dated raw results. Different endpoint from our creation trace.

ILLUSTRATION

Reusing state pays off when it avoids more setup than it adds.

Saved repeated setup $>$ save once + restart children + private changes

$$(N-1)P > H + N(R+D)$$

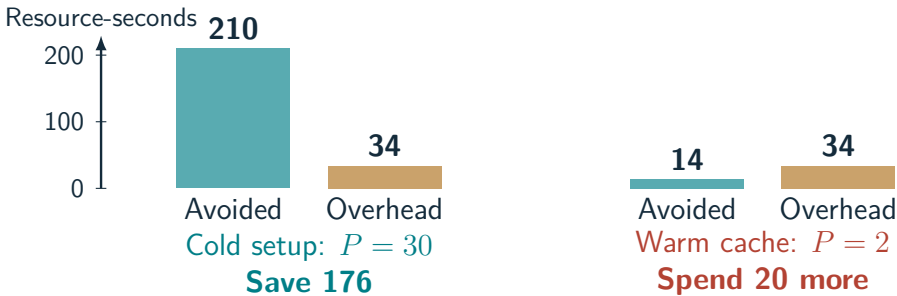
N	Number of alternative runs	H	Save the shared state once
P	Setup each run would repeat	R	Start a run from saved state
		D	Extra cost of private changes

Illustrative total-resource comparison. Common experiment work cancels; elapsed time also depends on scheduling.

ILLUSTRATION

Restoring wins against cold setup but loses against this warm cache.

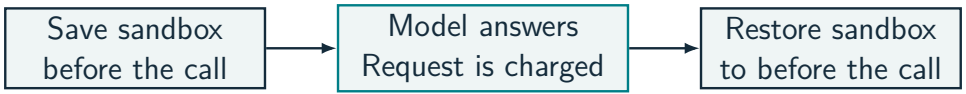
Eight alternative runs: $2 + 8(1 + 3) = 34$ resource-seconds of reuse overhead.



Illustrative additive resource model; $H = 2$, $R = 1$, $D = 3$ in both comparisons.

Restoring a sandbox does not undo an API call or its bill.

Example: a worker calls a hosted model after a checkpoint.

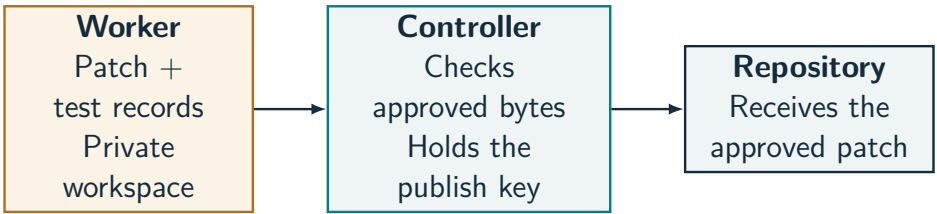


The external request has already happened.

Record the outcome outside the sandbox before deciding to retry.

Output-commit precedents: Speculator (2005), external synchrony (2006), Remus (2008).

Workers propose patches; the controller holds the key to publish them.

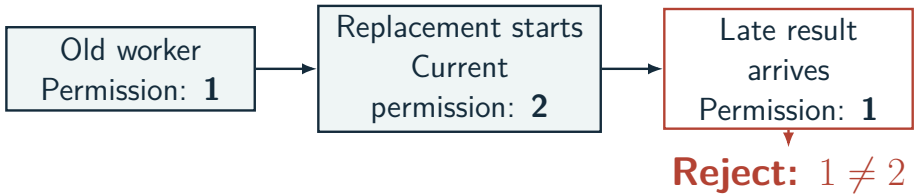


The worker can change its sandbox. Publication requires a separate decision.

Controller-held publication exercised in our factory; capability-security precedents: Capsicum and CHERI.

Once a worker is replaced, it can no longer publish results.

An **epoch** is the current version of a worker's permission.



The controller checks permission against its own current record.

Scheduling assigns work. This version check blocks outdated publication.

Proposed publication protocol; leases and fencing in distributed systems.

Combining branch results should add information while accounting for overlap.

Fan-out explores alternatives. **Fan-in** turns their results into a useful decision.

Choose or merge artifacts: check the final code, model or candidate.

Pool measurements: use their reliability and shared information.

Goal: reduce uncertainty without counting the same information twice.

Next: merge rules, overlapping observations, statistical weights and the resulting precision.

Choosing a result, merging code and pooling measurements are different operations.

Operation	Example	What is checked?
Select	Pick one of three model variants	The chosen model on evaluation data
Compose	Merge two code changes	The actual merged code
Pool	Combine estimates of one effect	Noise and information they share

The output determines the merge rule.

Factory artifact checks and the numerical reducer illustrate different return paths.

ILLUSTRATION

Code changes that pass in pairs can fail when all three are merged.

Example: each change starts a **1 GB worker**. The server has **2 GB**.

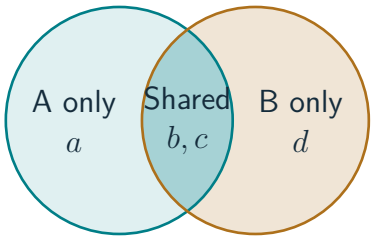
Merged changes	Memory used	Fits?
A, B or C alone	1 GB	✓ yes
A+B, A+C or B+C	2 GB	✓ yes
A+B+C	3 GB	× no

Run the checks on the final merged code before publishing it.

Illustrative resource-capacity example; successful pair tests do not cover every larger combination.

ILLUSTRATION

Combining results should keep shared observations once.



Union: 4 distinct observations

$$A \cup B = \{a, b, c, d\}$$

Intersection: 2 shared observations

$$A \cap B = \{b, c\}$$

New from B: 1 observation

$$B \setminus A = \{d\}$$

Different observations can still share errors. Mutual information describes statistical dependence.

Set diagram: exact illustrative observation IDs. Entropy identity: MIT information-theory notes.

The result type determines the fan-in operator.

How we fan out	Fan-in operator	What to check
Split one dataset	Union: $\bigcup_i A_i$	Coverage and duplicates
Try different ideas	Select: $x^\star = \arg\max_{x \in V} q(x)$	Same checks and score
Edit different components	$\mathrm{merge}(\Delta_1, \ldots, \Delta_k)$	Test the combined code
Repeat a measurement	Weighted average: $\sum_i w_i x_i$	$\sum_i w_i = 1$, shared noise
Seek common findings	Intersection: $\bigcap_i A_i$	Check agreement and gaps

A_i: returned items. V: candidates that pass checks. q: common score. Δ_i: edits from one base.

Max pooling returns the highest score. Argmax selects its candidate. ReLU clips negatives: $\mathrm{ReLU}(z) = \max(0, z)$. Agreement can reflect a shared error.

Our reducer catches duplicate observation IDs and keeps their source history.

Reducer: the code that combines returned numerical summaries.

Returned records	Reducer behavior
Observation 42 arrives twice	Reject the duplicate
Observations 42 and 43 arrive	Combine summaries; retain both IDs

Next step: use recorded shared sources to calibrate statistical weights.

Reference reducer: IDs and history are implemented; covariance-aware weights are the proposed extension.

For estimates of one quantity, covariance guides the pooling weights.

Combine estimates x_i using weights that sum to one: $\hat{\mu} = \sum_i w_i x_i$.

$$w = \frac{\Sigma^{-1}\mathbf{1}}{\mathbf{1}^T\Sigma^{-1}\mathbf{1}}$$

Σ: the error covariance matrix—how noisy estimates are and how their errors move together.

$\mathbf{1}$: a vector of ones. Independent errors: weights are proportional to $1/\sigma_i^2$.
Shared errors: use the full covariance matrix.

Unbiased estimates of a common target; known positive-definite covariance. Valassi & Chierici (2014), correlated measurement combination.

A worker that exaggerates its precision can dominate a weighted average.

$$\hat{\mu} = \frac{\sum_i p_i x_i}{\sum_i p_i}$$ p_i: precision, or inverse variance. Larger p_i means more weight.

Synthetic example: one 2,000-point worker targets 17 and inflates reported information per point $50\times$.



Reference demo: one synthetic stress case. The weighting rule needs trusted precision estimates.

ILLUSTRATION

With shared errors, 100 estimates can reduce noise like about nine independent ones.

Example: 100 estimates with equal noise and pairwise correlation $\rho = 0.1$.

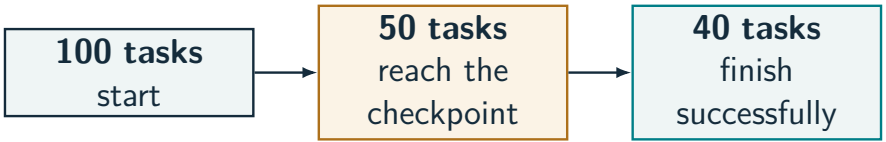
$$N_{\text{eff}} = \frac{N}{1 + (N-1)\rho} = \frac{100}{1 + 99(0.1)} = 9.17$$

	Independent	Correlation 0.1
Noise relative to one estimate	0.10σ	0.33σ
Signal-to-noise gain for fixed signal	$10\times$	$3.03\times$

Exact mean-variance illustration: same marginal noise σ, common pairwise correlation, unchanged signal.

ILLUSTRATION

If 50 of 100 tasks reach a checkpoint and 40 finish, overall success is 40%.



From the checkpoint: $40/50 = 80\%$. From the start: $40/100 = 40\%$.

Both rates are useful. State which starting population you are measuring.

Illustrative counts: $P(\text{success}) = P(\text{checkpoint})\,P(\text{success} \mid \text{checkpoint}) = 0.5 \times 0.8$.

A check record must say what ran, who ran it and which files it checked.

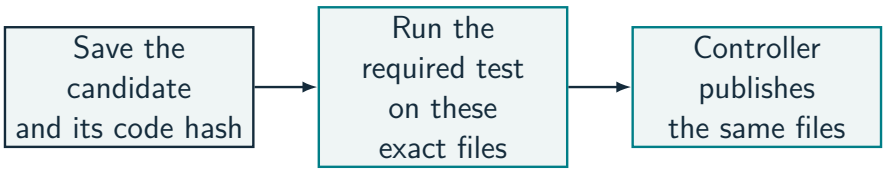
A **receipt** is a saved record of an operation and its result.

Label in our logs	What actually happened
CI job passed	Evaluation was skipped; job ended in **3 seconds**
Human approved	Automation answered the approval prompt
Code hash recorded	The helper code itself was unavailable

Keep the actual check result and the exact code needed to reproduce it.

Three audit findings from our factory. A code hash identifies bytes; it does not store them.

The code we publish must be the same code that passed the required check.



Repair example: make the workspace busy, reuse it for the same job, and assert **one dispatch**.

If the files change after testing, check the new version before publishing.

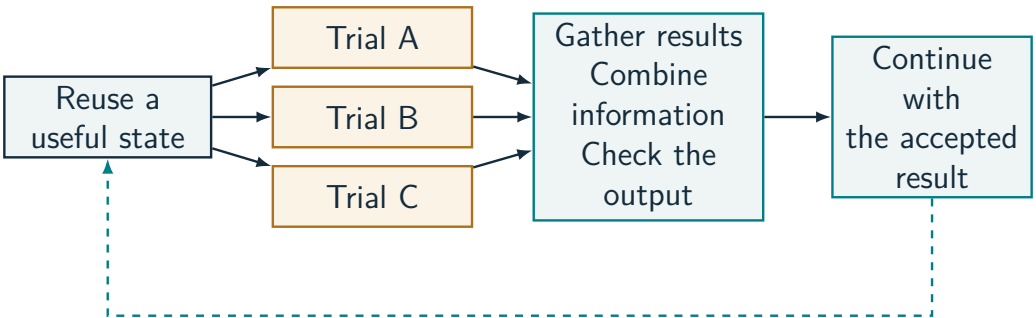
Proposed behavioral gate; builds on the factory's existing code-hash publication check.

Each use case needs a merge rule and a check matched to its output.

Use case	Gather / merge	Check before use
Model designs	Rank; select one	Evaluate the selected model
Code edits	Combine compatible edits	Test the merged code
Measurements	Remove duplicates; pool	Check noise and shared errors
Drug / orbit candidates	Rank or connect	Assays / numerical checks

Planned: restore versus warm reconstruction; siblings versus other families. Collection has not started.

Self-driving compute tries alternatives, combines what they teach us and continues.



Keep shared evidence once. Test the result. Let the controller accept it.

Forkable execution, information-aware aggregation and controller-held acceptance form the proposed system.

PUBLISHED ILLUSTRATION

Cheaper trials help when their results add useful evidence.

Operation	Numeric example	What it teaches
Reuse	**46 s → 13 s**	About 33 s saved per reported Tokio compile
Explore	**15.9% → 56%**	250 samples gave 3.5× candidate coverage
Aggregate	**100 → 9.17**	At correlation 0.1, count about nine independent equivalents

The goal is more useful, checked information for each unit of work.

Reuse: Tokio report. Explore: Brown et al., SWE-bench Lite coverage. Aggregate: equal-noise, common-correlation illustration.